

# Computer Architecture

CE/CS 321/330

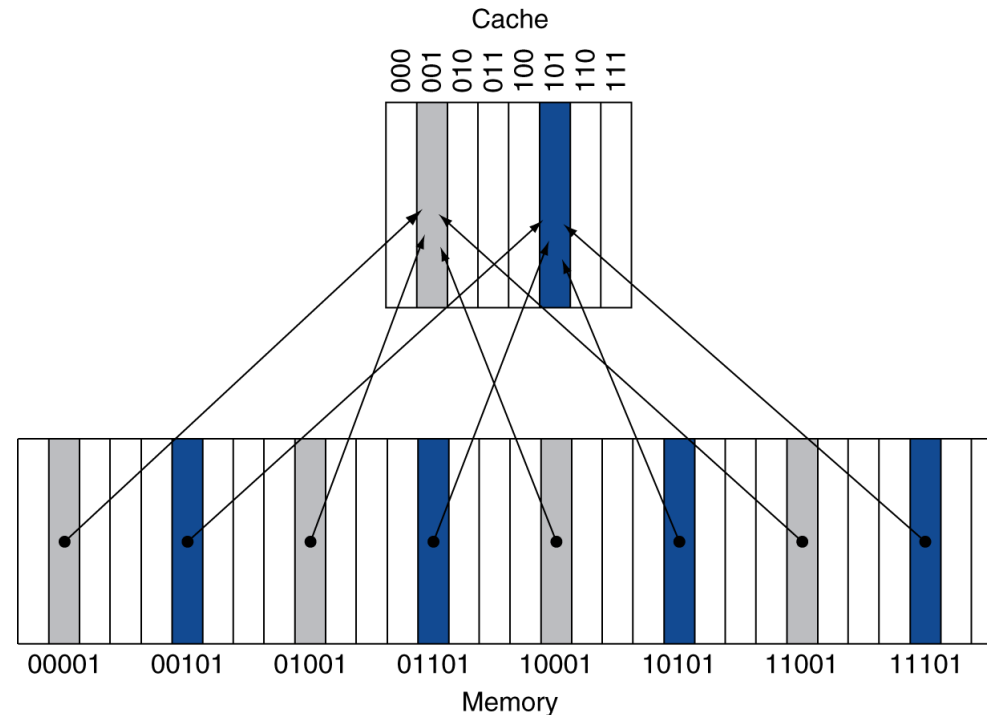
L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 25 + Quiz

# Direct Mapped Cache

- Direct Mapped Cache:
  - Assign a location in the cache for each word in memory based on the address of the word in memory
  - Each memory block (line) is mapped directly to exactly one line (block) in the cache.
  - The typical mapping between addresses and cache locations for a direct-mapped cache is usually simple, such as  
(Block address) modulo (#Blocks in cache)
  - If the number of entries in the cache is a power of 2, then modulo can be computed simply by using the low-order bits of the memory address.



# Tags and Valid Bits

- Because each cache location can contain the contents of a number of different memory locations, how do we know which particular MM block is stored in a cache line at a particular time?
  - Store block address as well as the data
  - Actually, only need the high-order bits of the address
  - Called the tag
- What if the cache line data is not valid?
  - Use a bit to differentiate between valid and not valid
  - Valid bit: 1 = present, 0 = not present
  - Initially 0

| Valid | Tag | Data |    |    |    | Index |
|-------|-----|------|----|----|----|-------|
| 1     | 12  | W1   | W2 | W3 | W4 | 00    |
| 0     | 43  | W1   | W2 | W3 | W4 | 01    |
| 1     | 05  | W1   | W2 | W3 | W4 | 02    |
| ⋮     | ⋮   | ⋮    |    |    |    |       |
| 1     | 56  | W1   | W2 | W3 | W4 | 63    |

# Cache Example

- 8-lines, 1 word/line, direct mapped
- Initial state

| Index | V | Tag | Data |
|-------|---|-----|------|
| 000   | N |     |      |
| 001   | N |     |      |
| 010   | N |     |      |
| 011   | N |     |      |
| 100   | N |     |      |
| 101   | N |     |      |
| 110   | N |     |      |
| 111   | N |     |      |

- Byte Addressable memory
- One word = 4 bytes
- Words are aligned, i.e. word starting addresses are divisible by 4
- Little Endian byte ordering

Word address can be calculated  
by multiplying word number by 4

word number x 4

# Cache Example

1

| Word num | Word Addr | Binary addr | Hit/miss | Cache line |
|----------|-----------|-------------|----------|------------|
| 22       |           |             |          |            |

| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | N |     |                |
| 001   | N |     |                |
| 010   | N |     |                |
| 011   | N |     |                |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | N | 10  | Mem[10 110 00] |
| 111   | N |     |                |

Word address can be calculated  
by multiplying word number by 4

word number x 4

|                |                |                |                |
|----------------|----------------|----------------|----------------|
| Mem[10 110 11] | Mem[10 110 10] | Mem[10 110 01] | Mem[10 110 00] |
| 91             | 90             | 89             | 88             |

# Cache Example

2

| Word num | Word Addr | Binary addr | Hit/miss | Cache line |
|----------|-----------|-------------|----------|------------|
| 26       | 104       | 11 010 00   | Miss     |            |

| Index      | V        | Tag       | Data                  |
|------------|----------|-----------|-----------------------|
| 000        | N        |           |                       |
| 001        | N        |           |                       |
| 010        | N        |           |                       |
| 011        | N        |           |                       |
| 100        | N        |           |                       |
| 101        | N        |           |                       |
| <b>110</b> | <b>Y</b> | <b>10</b> | <b>Mem[10 110 00]</b> |
| 111        | N        |           |                       |



| Index      | V        | Tag       | Data                  |
|------------|----------|-----------|-----------------------|
| 000        | N        |           |                       |
| 001        | N        |           |                       |
| <b>010</b> | <b>Y</b> | <b>11</b> | <b>Mem[11 010 00]</b> |
| 011        | N        |           |                       |
| 100        | N        |           |                       |
| 101        | N        |           |                       |
| 110        | Y        | 10        | Mem[10 110 00]        |
| 111        | N        |           |                       |

# Cache Example

3  
4

| Word num | Word Addr | Binary addr | Hit/miss | Cache line |
|----------|-----------|-------------|----------|------------|
| 22       | 88        | 10 110 00   | Hit      | 110        |
| 26       | 104       | 11 010 00   | Hit      | 010        |

| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | N |     |                |
| 001   | N |     |                |
| 010   | Y | 11  | Mem[11 010 00] |
| 011   | N |     |                |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | Y | 10  | Mem[10 110 00] |
| 111   | N |     |                |

# Cache Example

|   | Word num | Word Addr | Binary addr | Hit/miss | Cache line |
|---|----------|-----------|-------------|----------|------------|
| 5 | 16       | 64        | 10 000 00   |          | 000        |
| 6 | 3        | 12        | 00 011 00   |          | 011        |
| 7 | 16       | 64        | 10 000 00   |          | 000        |

| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | N |     |                |
| 001   | N |     |                |
| 010   | Y | 11  | Mem[11 010 00] |
| 011   | N |     |                |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | Y | 10  | Mem[10 110 00] |
| 111   | N |     |                |



| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | Y | 10  | Mem[10 000 00] |
| 001   | N |     |                |
| 010   | Y | 11  | Mem[11 010 00] |
| 011   | Y | 00  | Mem[00 011 00] |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | Y | 10  | Mem[10 110 00] |
| 111   | N |     |                |

# Cache Example

8

| Word num | Word Addr | Binary addr | Hit/miss | Cache line |
|----------|-----------|-------------|----------|------------|
| 18       | 72        | 10 010 00   |          | 010        |

| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | Y | 10  | Mem[10 000 00] |
| 001   | N |     |                |
| 010   | Y | 11  | Mem[11 010 00] |
| 011   | Y | 00  | Mem[00 011 00] |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | Y | 10  | Mem[10 110 00] |
| 111   | N |     |                |



| Index | V | Tag | Data           |
|-------|---|-----|----------------|
| 000   | Y | 10  | Mem[10 000 00] |
| 001   | N |     |                |
| 010   | Y | 10  | Mem[10 010 00] |
| 011   | Y | 00  | Mem[00 011 00] |
| 100   | N |     |                |
| 101   | N |     |                |
| 110   | Y | 10  | Mem[10 110 00] |
| 111   | N |     |                |

# Computer Architecture

CE/CS 321/330

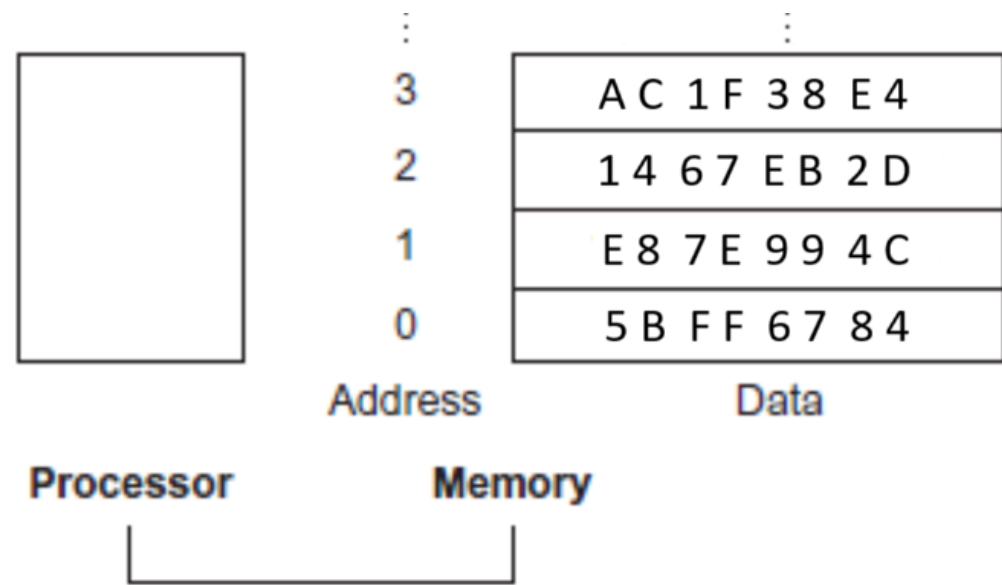
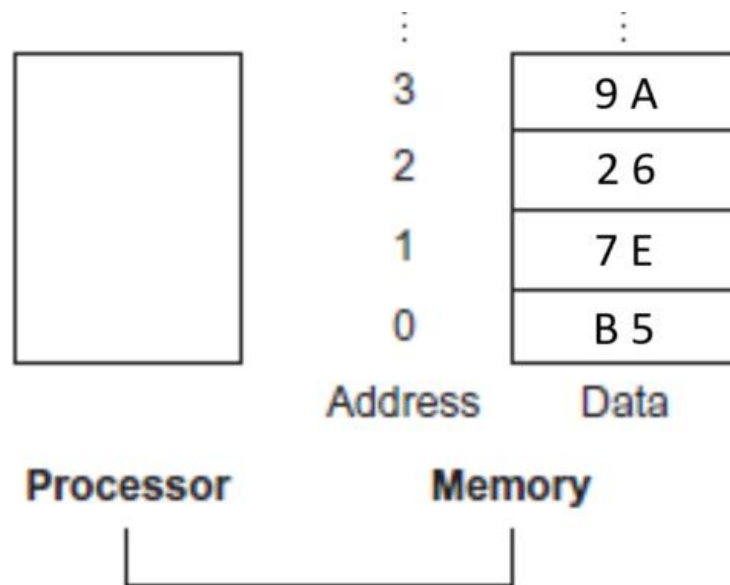
L2

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture: 26

# Memory Addressing

- How many bytes does each address identify
  - In RISC-V, each memory address identifies one byte (8 bits)



# Example: Direct mapped Cache

- Consider a direct mapped cache having 64 lines with 8 words/line and word = 4 bytes, addressable unit is a byte
  - To what cache line number (cache block number) does word address 12200 map?
  - If MM address is of 20 bits what is the tag size?

| Cache size         | block (line) size  | # cache lines      | MM size        | # MM blocks     |
|--------------------|--------------------|--------------------|----------------|-----------------|
| 64 x 8 = 512 words | 8 words            | 64 lines ( $2^6$ ) | $2^{20}$ bytes | $2^{15}$ blocks |
| 2K bytes (2 KiB)   | 32 bytes ( $2^5$ ) |                    | $2^{18}$ words |                 |

- Ans a)
  - First find the MM Block number to which address 12200 belongs
  - Since memory is byte addressable address 12200 will be in block number
    - MM block number =  $\lfloor 12200/32 \rfloor = \lfloor 381.25 \rfloor = 381$
  - Next we can find the cache line number using the formula
    - Cache line number = MM block number (modulo) # cache lines
    - Cache line number =  $381 \text{ modulo } 64 = 61$

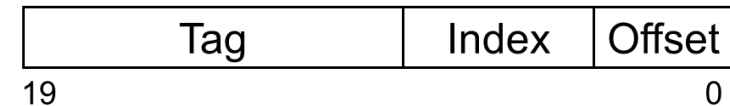
# Example: Direct mapped Cache

b) If MM address is of 20 bits what is the tag size?

| Cache size         | block (line) size  | # cache lines      | MM size        | # MM blocks     |
|--------------------|--------------------|--------------------|----------------|-----------------|
| 64 x 8 = 512 words | 8 words            | 64 lines ( $2^6$ ) | $2^{20}$ bytes | $2^{15}$ blocks |
| 2K bytes (2 KiB)   | 32 bytes ( $2^5$ ) |                    | $2^{18}$ words |                 |

• Ans b)

- First let's determine how will the cache interpret the MM address of 20 bits
- Cache separates this address into three fields Offset, Index and Tag



- No. of bits for Offset or Word = ? (How many addressable units are there in a block?)(bits to distinguish each of these addressable unit)

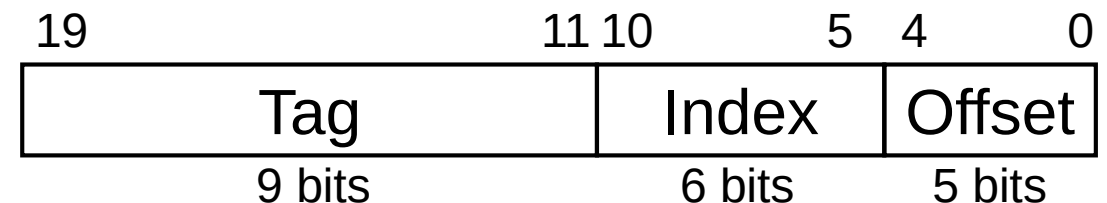
- $Offset\ bits = \log_2(block\ size)$
- 5

- No. of bits for Index or line = ? (How many cache lines are there?)(bits to distinguish these cache lines)

- $Index\ bits = \log_2(\#cache\ lines)$
- 6

- No. of bits for tag field = ? (How many blocks are contending for the same line?)(bits to distinguish these contending blocks)

- $20 - 5 - 6 = 9\ bits$  (easy way) or
- $Tag\ bits = \log_2\left(\frac{\# MM\ blocks}{\# Cache\ lines}\right) = \log_2\left(\frac{2^{15}}{2^6}\right)$

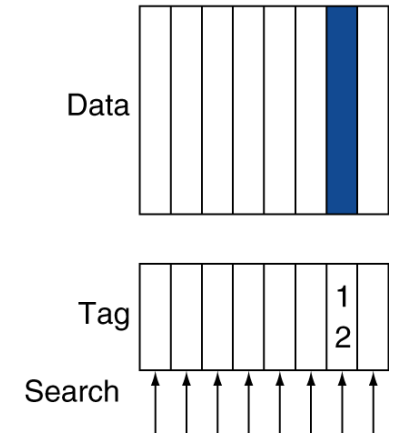


# Associative Caches

- Fully Associative Cache

- Allows a given block to go in any cache entry
- To find a block in the cache, all entries in the cache must be searched.
- Because every cache line has to be checked this significantly increases the hardware cost
- However, will result in lower miss rate because of more options for MM blocks to be placed

**Fully associative**



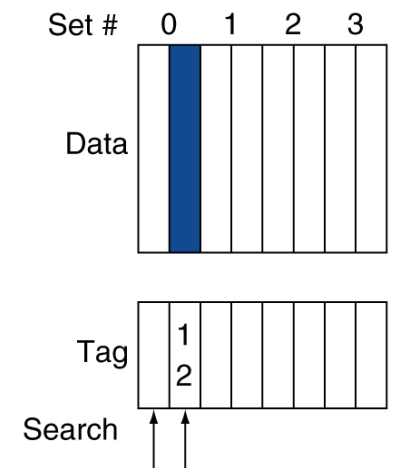
- Set Associative Cache

- An n-way set-associative cache consists of a number of sets, each of which consists of n lines.
- Each block in the memory maps to a unique set in the cache, and a block can be placed in any line of that set

$(\text{Block number}) \bmod (\text{\#Sets in cache})$

- To find a block in the cache, all entries in a given set need to be searched.
- Less expensive in terms of hardware as compared to fully associative cache

**Set associative**



# Spectrum of Associativity

- For a cache with 8 entries:

## One-way set associative (direct mapped)

| Block | Tag | Data |
|-------|-----|------|
| 0     |     |      |
| 1     |     |      |
| 2     |     |      |
| 3     |     |      |
| 4     |     |      |
| 5     |     |      |
| 6     |     |      |
| 7     |     |      |

## Two-way set associative

| Set | Tag | Data | Tag | Data |
|-----|-----|------|-----|------|
| 0   |     |      |     |      |
| 1   |     |      |     |      |
| 2   |     |      |     |      |
| 3   |     |      |     |      |

## Four-way set associative

| Set | Tag | Data | Tag | Data | Tag | Data | Tag | Data |
|-----|-----|------|-----|------|-----|------|-----|------|
| 0   |     |      |     |      |     |      |     |      |
| 1   |     |      |     |      |     |      |     |      |

## Eight-way set associative (fully associative)

| Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data |
|-----|------|-----|------|-----|------|-----|------|-----|------|-----|------|-----|------|-----|------|
|     |      |     |      |     |      |     |      |     |      |     |      |     |      |     |      |

# Memory Hierarchy

- Four Memory Hierarchy Questions
  - Q1: Where can a block be placed in the upper level? (block placement)
  - Q2: How is a block found if it is in the upper level? (block identification)
  - Q3: Which block should be replaced on a miss? (block replacement)
  - Q4: What happens on a write? (write strategy)

# Direct Mapped Cache: Finding a Block in Cache

32-bit Addresses

Addresses for single byte

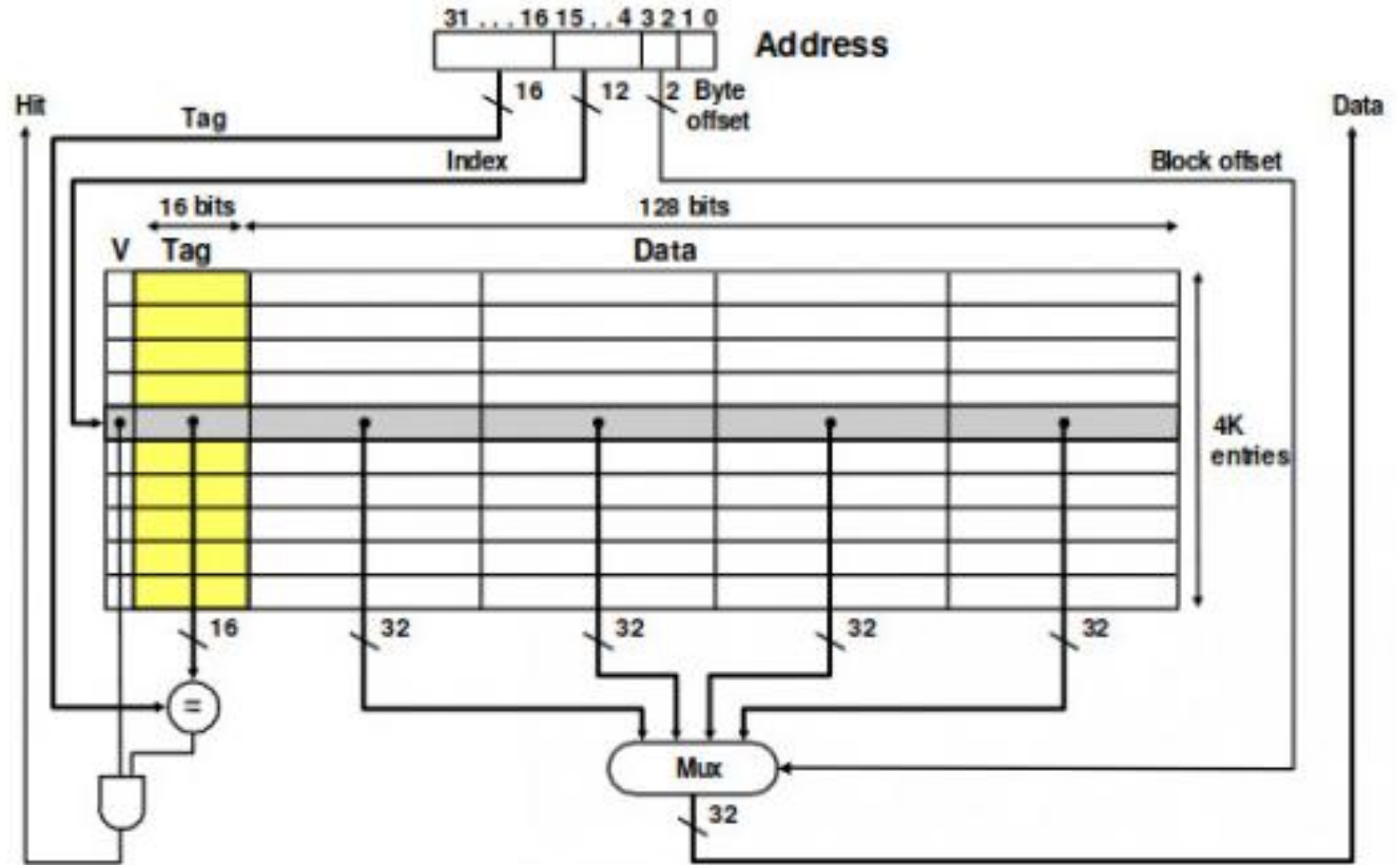
Block Size:  $2^2 = 4$  words (16 bytes)

Cache Size:  $2^{16}$  bytes

$2^{12}$  lines (blocks)

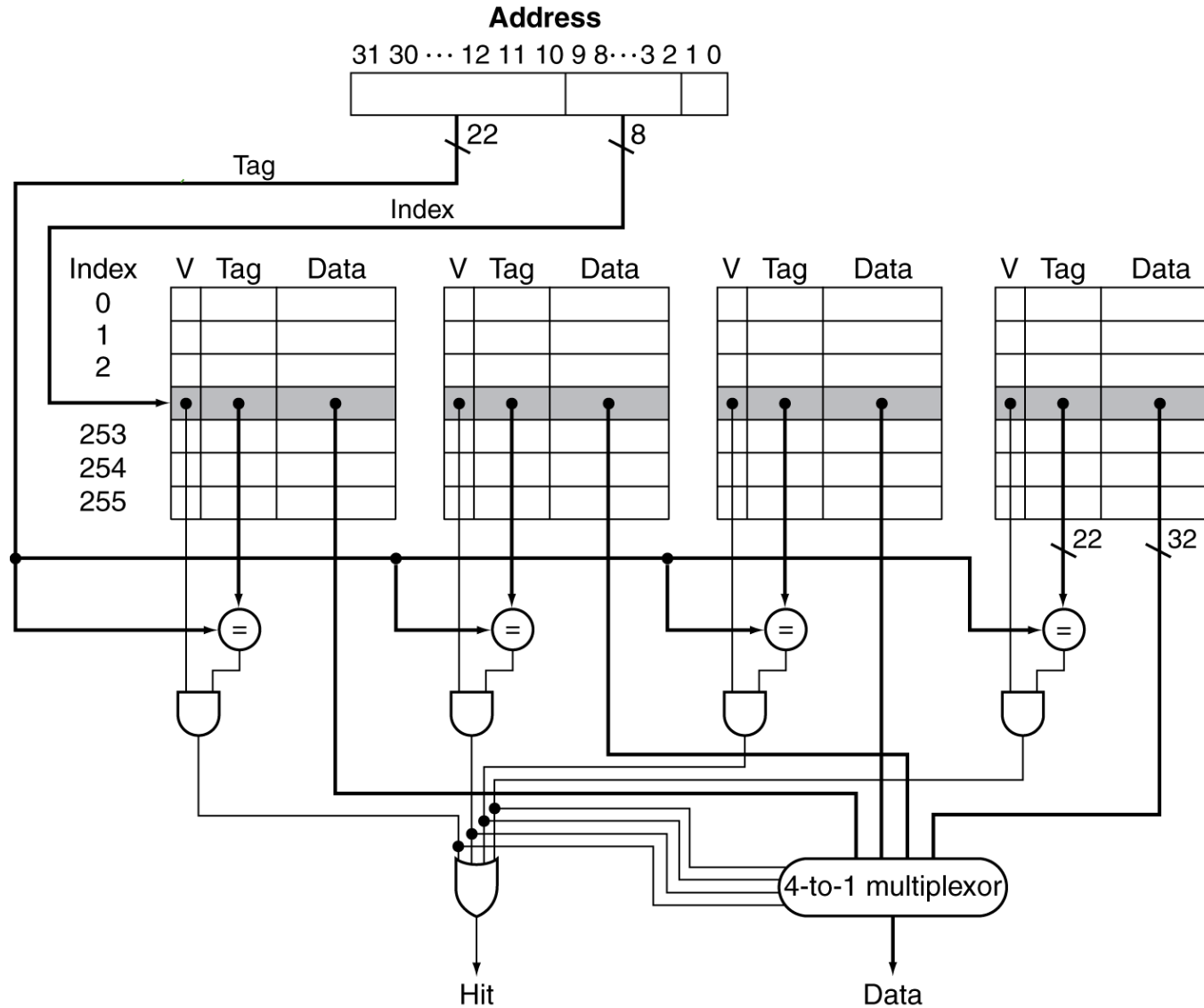
MM Size:  $2^{32}$  bytes

MM Blocks:  $2^{28}$  blocks



# 4-Way Set Associative Cache: Finding a Block in Cache

Block size = 1 word



# Cache Size

- How many total bits are required for a direct-mapped cache with 16 KiB of data and four-word blocks, assuming a 64-bit address?